CS 4265: Big Data Analytics
CollegeFootballDB: A Distributed Analytics Platform for SEC Play by Play Data

Executive Summary

What does this pipeline do:

CollegeFootballDB is a distributed analytics platform that ingests, transforms, and allows queries for SEC college football play by play data from 2021 to 2025. The pipeline fetches raw JSON data from the CollegeFootballData.org API, stores it in Amazon Web Services S3, converts it to Snappy compressed Parquet via PySpark schema-enforcement job, and then runs a MapReduce aggregation to produce per game offensive statistics for all 14 SEC teams across five seasons. Results are queryable through Spark SQL.

What problem does it solve:

Traditional single machine tools like pandas are inefficient for complex aggregations and joins across tens of thousands of plays and multiple seasons. Analysts and coaches also want to answer questions such as “How has offensive play calling changed over the last five years?” or “Which SEC offense is producing the most big plays?” These questions require a system that can quickly process hundreds of thousands of plays, enforce schema consistency, and deliver sub second query response times. CollegeFootballDB provides that infrastructure using distributed storage (S3), parallel processing (Spark), and a query interface (SQL).

Key results:

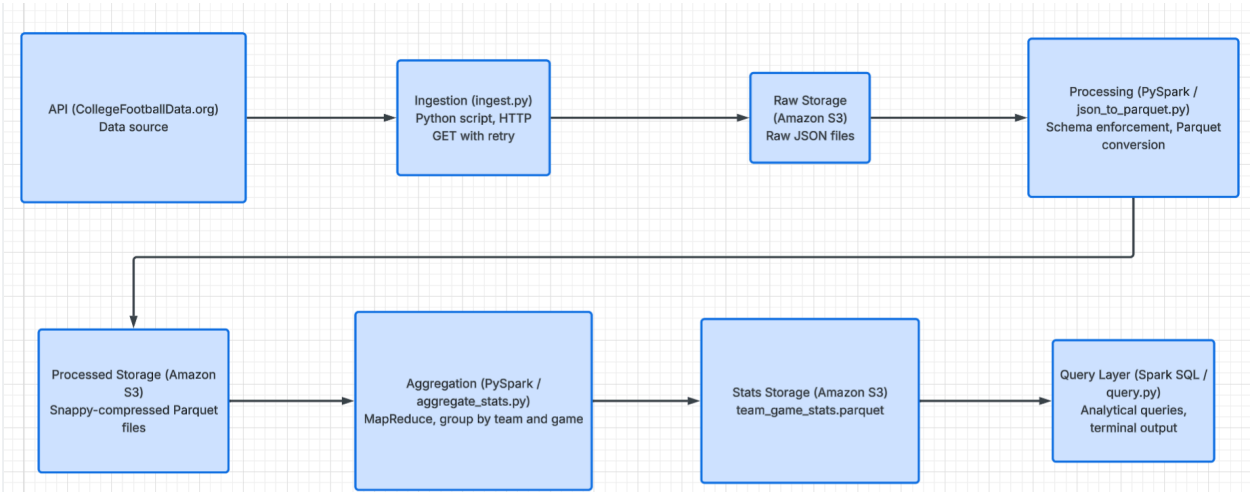
151,344 plays ingested across 14 SEC teams, 5 seasons (2021-2025), producing 1,158 game statistic rows.

Full pipeline from Ingestion to MapReduce aggregation runs in under 8 minutes.

Parquet and Snappy compression delivers an estimated 3-5x storage reduction when compared to raw JSON. Columnar predicate pushdown also allows for faster analytical queries.

Architecture

Final architecture diagram:



Technology stack with justifications:

Technology	Role in Pipeline	Rationale
Amazon Web Services S3	Storage	Cloud storage for all raw JSON and processed Parquet files. Spark can read directly from S3.
Apache Spark 4.0	Distributed Processing	Parallelizes processing across a cluster to handle large datasets quickly.
Apache Parquet + Snappy	File Format	Parquet's columnar layout lets queries read only the columns they need, and Snappy provides fast compression. Together they cut I/O and speed up analytical queries.
Spark SQL	Query interface	Spark SQL lets users query using regular SQL that many people already know. It automatically figures out the most efficient way to run the query before it touches any data, so optimization from the

		user is not needed.
Python-dotenv / .gitignore	Credential management	Keeps API keys and AWS secrets out of the public repository.
Hadoop	Credential management	Hadoop's S3A connector handles AWS authentication via IAM roles or env variables, so Spark can access S3 securely.

Data flow description:

Stage 1: Ingestion.py is run which first authenticates with the CollegeFootballData.org API using an educational API key. For each SEC team, season, and week, it issues an HTTP GET with exponential retry/backoff to handle rate limit errors (429). Raw JSON responses are written to S3 under the path raw/year=YYYY/week=W/{Team}_plays.json. Before calling the API, the script will check if the file already exists in S3 so API calls are not wasted.

Stage 2: JSON to Parquet conversion is done using json_to_parquet.py. A PySpark job reads all raw JSON from S3, enforces an explicit 27 column typed schema, filters corrupt records into a corrupt_record column that is then logged and dropped, and writes Snappy compressed Parquet files partitioned by season and week to s3a://{bucket}/processed/season=YYYY/week=W/.

Stage 3: MapReduce Aggregation is done using aggregate_stats.py. A second Spark job is made which implements the MapReduce pattern. The map phase filters to scrimmage only plays which removes kickoffs and punts. Per play success is calculated, third down attempts and conversions are tagged, and big plays are marked. The reduce phase groups by gameId, team, and opponent to produce per game offensive statistics. Output is written to s3a://{bucket}/stats/team_game_stats.parquet.

Query Interface: Spark SQL queries are run against the processed Parquet files and stats output with results printed to the terminal.

Implementation

Key technical decisions and rationale:

Decision 1: The first decision I made was choosing S3 over HDFS for storage. Using S3 eliminates the operational overhead of managing HDFS DataNodes and NameNode, while still providing the distributed and fault tolerant storage necessary. Spark is also able to read and write to S3 objects directly.

Decision 2: Another decision that was made was choosing Explicit Schema Enforcement over Schema Inference. Rather than allowing Spark to infer types from JSON which can cause multiple issues, an explicit 27 column schema was defined in json_to_parquet.py. This catches

type errors at ingestion time, ensures downstream aggregations operate on correctly typed columns, and places corrupted records in a logged corrupt_record column rather than silently dropping or miscasting them.

Stack concepts applied:

Concept	Application
MapReduce Pattern	Aggregate_stats.py implements map (filter plays, compute per play metrics, and tag 3rd downs and big plays) and reduce (groupBy gameId + team + opponent, aggregate yardage, success rate, and conversion rates) using the Spark DataFrame API.
Distributed Storage	Raw JSON and Parquet files are partitioned by season and week in S3, mirroring GFS block distribution. Partitioning allows Spark to skip irrelevant S3 prefixes during queries which is similar to DataNode locality in HDFS.
Columnar Storage	JSON is converted to Parquet with snappy compression in Stage 2. A query like SELECT AVG(yardsGained) WHERE down=3 will only read the yardsGained and down columns.

Challenges overcome:

Challenge 1: A challenge that I ran into quickly was exhausting my API calls. The API's free plan only allowed me to get one quarter of the amount of data needed for my pipeline. I solved this by doing some more research on the API and I found that they offered an educational API which would give me enough calls to implement the project. I also adjusted my ingestion script to check if the data already exists in S3, so I wouldn't waste any additional API calls on data that already exists.

Challenge 2: Another issue that I had was getting rate limited from the API. After running my ingestion script for a few seconds I noticed I was getting errors and data was no longer being ingested. This was because the API would limit you rather quickly. I solved this by adding a configurable delay between API calls which brought the rate limit rate to zero.

Challenge 3: I also ran into issues with the Java version needed for Spark. While developing on Windows, I was getting errors with Spark saying that it could not detect the right version. I temporarily solved this for M3 by explicitly mentioning my Java 17 filepath. I later implemented a permanent fix by setting the JAVA_HOME path in Windows PowerShell to where Java 17 is installed (this is only needed if you have multiple Java versions installed on a device).

Validation

Data quality metrics:

Metric	Value	Notes
Total plays ingested	151,344	14 SEC teams x 5 seasons
Game stat rows produced	1,158	Output of MapReduce aggregation
Schema columns enforced	27	Explicit typed schema in Stage 2
Seasons covered	5	5 complete seasons
Teams converted	14	Texas and Oklahoma excluded because they joined in 2024
Corrupt / rejected records	0	Logged to corrupt_record column
Null rate	25.11%	Kickoffs and returns have null ppa by design
Duplicate records	0	S3 is checked during ingestion to prevent duplicates
Parquet compression ratio	9.35x	Snappy vs raw JSON

Validation approach and evidence:

Stage 1: After ingestion, the total plays retrieved was confirmed to be 151,344. A sample run for 2024 week 3 returned 2,514 rows which was consistent with the expected 180 plays per game x multiple games.

Stage 2: By enforcing an explicit 27 column schema, any record that does not conform is captured in corrupt_record rather than being allowed to corrupt the dataset. The presence of this column and its row count provides a data quality signal.

Stage 3: The 1,158 game stat rows produced by the aggregation are consistent with the expected volume.

Query: When querying for top SEC offensive performances in terms of yards per play, the top result is Abilene Christian University vs. Missouri and there were 14 yards per play. This result makes sense because it is a small FCS football team against an SEC FBS football team.

Performance characteristics:

Stage 1: 162.4 seconds
Stage 2: 335.5 seconds
Stage 3: 22.1 seconds

The total pipeline runs in 8.6 minutes.

Reflection

What would I do differently:

One thing I would do differently is that I would try and find another API provider. Unfortunately, this API only gave me so many free calls so I was limited to doing just 5 seasons across one conference. If I wanted to scale my project and include more seasons or conferences, I would have to pay a lot of money for continued API access. I would also want to look into using a different version of Spark. I ran into some compatibility issues when developing on Windows. It required me to use a specific Java version that had to be downloaded from a specific website. Lastly, I would implement a better pipeline orchestration tool such as Apache Airflow or Prefect instead of using run_pipeline.py. As my pipeline grew, having a better pipeline orchestration tool could have made dependency management, monitoring, and reruns cleaner and more efficient.

What did I learn:

I learned how to manage and use an API. Running out of calls or getting rate limited is a real world issue so I now know what to do if I encounter those issues again. I also have a much deeper understanding of why columnar storage formats like Parquet exist. The performance difference between running an analytical query against raw JSON vs. Parquet is substantial. I also better understand Git and creating a professional GitHub repo.

Future improvements if continued:

Performance: Profile and optimize the Parquet conversion stage. Increase executor memory allocation and add better input partition sizing. I would target at least a 50% decrease in time needed to run stage 2.

Visualizations: A visualization layer could be added to render trends such as yards per play by season, third down conversion rates by team, and play type distribution over time. This would allow for more insights to be generated by analysts and coaches.

Scale: If I had access to more API calls, I could increase the scale of the project to cover more seasons, more conferences, and maybe even multiple divisions.